IEEE IAS SLIIT Website: Setup Guide

How to set up this project from scratch on a new Supabase project.

0. Prerequisites

Tool	Version	Used for
Node.js	20 or newer (CI uses 20)	Running and building the site
Git	any	Cloning the repo
Supabase account	free plan is fine	Database, auth, email
Google account	2-Step Verification turned on	Sending email through Gmail SMTP
Vercel account (optional)	n/a	Hosting
Supabase CLI (optional)	latest	Running migrations from the terminal

1. Clone and install

```
git clone https://github.com/PulithThewmika/IEEE-SLIIT-IAS-Web-Clone.git
cd IEEE-SLIIT-IAS-Web-Clone
npm install
```

2. Create the Supabase project

1. Go to https://supabase.com/dashboard and click **New project**.
2. Choose a name and region (Singapore, `ap-southeast-1`, is closest to Sri Lanka) and a strong database password. Save the password; you need it for the CLI.
3. Wait about 2 minutes for the project to finish setting up.

3. Environment variables (`.env`)

In the Supabase dashboard, open **Project Settings** → **API** (or the **Connect** button) and copy two values:

- **Project URL**
- **anon / publishable key**. Use only this key. The `service_role` key bypasses all security and must never go in the frontend.

Copy the template and fill it in:

```
cp .env.example .env
```

```
VITE_SUPABASE_URL=https://xxxxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOi...
```

`.env` is gitignored, so don't commit it. If these values are missing, the site loads as a **blank white page**.

> **Important:** SMTP credentials do **not** go in `.env`. Every `VITE_` variable is bundled into the browser JavaScript, where anyone can read it. Email is sent by Supabase, so the SMTP settings go in the Supabase dashboard (step 6).

4. Run the database migrations

The files in `supabase/migrations/` must run **in filename order**:

#	File	What it does
1	`20260731000000_init_schema.sql`	Core tables, admin `profiles`, `is_admin()`, row-level security (RLS)
2	`20260808235900_add_gallery_photographer.sql`	Gallery photographer column
3	`20260809000000_mock_seed_data.sql`	**Sample data (optional)**
4	`20260809010000_member_system.sql`	Member registrations, news items
5	`20260809020000_mock_news_seed.sql`	**Sample news (optional)**
6	`20260810000000_security_hardening.sql`	Signup trigger fix, moderation, magazine table, URL checks
7	`20260810010000_newsletter_articles.sql`	Member article submissions
8	`20260810020000_repair_seed_encoding.sql`	Fixes garbled characters in sample data (harmless without it)

Option A: SQL Editor (no install). Open each file in order, paste it into Dashboard → **SQL Editor** → **New query**, and click **Run**. Each one should finish with "Success". If one fails, stop there and fix it before running the next.

Option B: Supabase CLI

```
npx supabase login
npx supabase link --project-ref <your-project-ref>   # the ref is the subdomain in your Project URL
npx supabase db push
```

To start without sample data, delete files 3 and 5 before you run the migrations.

5. Create the first admin

Signing up from the website always creates a *member* account, never an admin. This is a deliberate security measure. To create an admin:

1. Go to Dashboard → **Authentication** → **Users** → **Add user** → **Create new user**. Enter the email and password and tick **Auto Confirm User**.
2. In the **SQL Editor**, run:

```
INSERT INTO public.profiles (id, full_name)
SELECT id, 'Admin Name' FROM auth.users WHERE email = 'admin@example.com';

DELETE FROM public.member_registrations
WHERE id = (SELECT id FROM auth.users WHERE email = 'admin@example.com');
```

3. Log in at `/admin/login`.

6. Gmail SMTP (sends the confirmation emails)

Supabase's built-in email sender is for testing only. It allows just a few emails per hour and only to your own team, so set up custom SMTP.

6a. Get a Google App Password

1. Sign in to the Gmail account you'll send from (ideally a club account such as `ias.sliit@gmail.com`).
2. Turn on **2-Step Verification** at https://myaccount.google.com/security. App passwords don't appear until this is on.
3. Go to https://myaccount.google.com/apppasswords.
4. Enter an app name such as `Supabase IAS` and click **Create**.
5. Copy the **16-character password**. Google shows it only once. Remove the spaces when you paste it.

If you see "The setting you are looking for is not available", the cause is one of: 2-Step Verification isn't on, the account is a Workspace account whose admin has turned app passwords off, or the account uses Advanced Protection.

6b. Enter the settings in Supabase

Go to Dashboard → **Authentication** → **Emails** → **SMTP Settings** and turn on **Enable Custom SMTP**:

Field	Value
Sender email	`ias.sliit@gmail.com` (must be the same Gmail account)
Sender name	`IEEE IAS SLIIT`
Host	`smtp.gmail.com`
Port	`465` (SSL), or `587` (STARTTLS) if 465 is blocked
Username	the full Gmail address
Password	the 16-character app password, without spaces
Minimum interval	`60` seconds (the default)

Click **Save**.

6c. Rate limits

- Under **Authentication** → **Rate Limits**, the email-sending limit starts at 30 per hour once custom SMTP is on. Raise it if you expect many signups at once.
- Gmail itself allows about **500 emails per day** from a personal account. That's fine for a student chapter. If you outgrow it, move to Resend, Brevo or SendGrid; only the host, port and credentials change.

7. Customize the email template

1. Go to Dashboard → **Authentication** → **Emails** → **Templates** → **Confirm signup**.
2. **Subject:** `Confirm your IEEE IAS SLIIT account`
3. **Body:** paste the whole of `supabase/email-templates/confirmation.html`.
4. Keep the placeholders exactly as written: `{{ .ConfirmationURL }}` and `{{ .Email }}`.
5. To change it: the brand colours are navy `#0f2438` and green `#007a3d`. Edit the text between the tags. Use only inline styles and web-safe fonts, because email clients strip `<style>` blocks and web fonts.
6. Click **Save**, then sign up with a test address to check it.

Other templates (Magic Link, Reset Password, Change Email, Invite) aren't used by the site yet. You can leave them as they are.

8. Auth settings

- **Authentication** → **Sign In / Providers** → **Email:** turn on **Enable Email provider** and **Confirm email**.
- **Authentication** → **URL Configuration:**
  - **Site URL:** your live domain, e.g. `https://ias-sliit.vercel.app`. The confirmation link sends users here, so leaving it at `localhost` breaks the links for real users.
  - **Redirect URLs:** add `http://localhost:5173/**` and `https://<your-domain>/**`, plus `https://*-<your-vercel-team>.vercel.app/**` if you use Vercel preview deploys.

9. Run locally

```
npm run dev
```

Open **http://localhost:5173**. If that port is busy, Vite moves to 5174, 5175 and so on, so check the terminal for the actual URL. To pin the port, run `npm run dev -- --port 3000`. To test from your phone on the same Wi-Fi, run `npm run dev -- --host`.

Command	Port	Purpose
`npm run dev`	5173	Development server with hot reload
`npm run build`	n/a	Production build into `dist/`
`npm run preview`	4173	Serves the built `dist/` locally
`npm run lint`	n/a	oxlint

To stop a server that's still holding a port on Windows (PowerShell):

```
Get-NetTCPConnection -LocalPort 5173 -State Listen | % { Stop-Process -Id $_.OwningProcess -Force }
```

10. Deploy to Vercel

1. Go to https://vercel.com, click **Add New** → **Project** and import the GitHub repo. Vercel detects Vite automatically, and `vercel.json` already handles the SPA rewrites and security headers.
2. Under **Settings** → **Environment Variables**, add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` for Production and Preview.
3. Click **Deploy**, then put the live URL into the Supabase **Site URL** (step 8).

11. GitHub Actions (CI)

In the repo, go to **Settings** → **Secrets and variables** → **Actions** and add:

Secret	Needed by	Where to get it
`VITE_SUPABASE_URL`	`ci.yml` (build check)	Step 3
`VITE_SUPABASE_ANON_KEY`	`ci.yml`	Step 3
`VERCEL_TOKEN`	`vercel-deploy.yml` (optional)	Vercel → Account Settings → Tokens
`VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`	`vercel-deploy.yml`	`.vercel/project.json` after running `npx vercel link`

If you deploy through Vercel's own Git integration, skip the Vercel secrets. Without them, the workflow skips itself.

12. Day-to-day admin

- **Approving members:** a new signup confirms their email and then stays `pending` until an admin activates them in the admin panel. Until then they can't log in.
- **Content:** events, gallery, newsletter issues, magazine editions and the member wall are managed in `/admin`. Images and PDFs are referenced by URL. The project has no Supabase Storage buckets, so host files in `public/` or on any HTTPS URL.
- **Forgotten passwords:** the site has no password-reset page yet. An admin can reset a password from Dashboard → Authentication → Users.

13. Troubleshooting

Symptom	Cause	Fix
Blank white page	`.env` missing or empty	Step 3, then restart `npm run dev`
No confirmation email arrives	SMTP misconfigured, or the email rate limit was hit	Check Dashboard → Logs → Auth, re-check the app password and port
`535 Username and Password not accepted`	You used the normal Gmail password	Use the 16-character app password
Confirmation link opens localhost	Site URL not set	Step 8
"Account is not yet activated"	Waiting for admin approval	An admin activates the member
Admin login says "no admin access"	No row in `profiles`	Step 5
Odd characters like `ΓÇö` in text	Text was pasted through the Windows clipboard	Re-type the text, or run migration 8
Port 5173 is in use	An old dev server is still running	Stop it (step 9)

14. Security checklist

- Never commit `.env`, and never put the `service_role` key or the SMTP password in any `VITE_` variable.
- Create admins only through SQL (step 5).
- Keep row-level security (RLS) on for every table. All the migrations already turn it on.
- If an app password leaks, revoke it at myaccount.google.com/apppasswords and create a new one.